Autonomous UAV-Based RF Source Detection and Localization Using RSSI

Dhruv Nikalwala

Computer Engineering
The University of British Columbia

August 21, 2026,

Introduction:

The purpose of this project is the detection and estimation of GPS coordinates of an unknown RF source. A UAV is useful for this application as it can be automated to fly in a pattern in open search areas. UAV has the benefit of controlled speed with the ability to manoeuvre smoothly maintaining a certain altitude and stop or hover over an area when required.

The overall flow of an autonomous drone is that it takes off then follows the zig zag pattern while continuously checking the *Received Signal Strength Indicator (RSSI)* values. Once it detects a predetermined RSSI threshold the drone stops. After that, it activates the circle mode which records the RSSI values with the corresponding X and Y positions in a circular pattern with a certain radius, around the detected region. Later, when it comes back to the starting point of the circle mode it will calculate the estimated GPS position based on the collected data. After that, it will return to the home location completing the autonomous mission.

Project Objectives and Requirements:

The main objective of the project was to develop an autonomous UAV system that is able to detect RF source and the estimated GPS location of it.

- Initialize connection between Raspberry PI and Pixhawk flight controller.
- Take off and maintain a certain altitude.
- Follow the dedicated zig zag pattern autonomously.
- Continuously receiving the RSSI value.
- Detect when the RSSI value passes a predetermined threshold.
- Stop and advance into circle mode.
- Collect RSSI values with the Corresponding local X and local Y values and save it.
- Estimate the RF source location with the collected values in local X and local Y .
- Convert that local X and Y to Global Positioning System coordinates.
- Return to the saved home location.

The Raspberry Pi was used as the companion computer for running the autonomous mission logic, while the Pixhawk handled flight control and stabilization.

Hardware and Architecture:

- **F450 quadcopter/frame:** This is the drone frame we are using. It is made from nylon and fiber, which is very light, strong and durable. The drone frame also includes Power distribution PCB which distributes power to the 4 ESC's and Pixhawk.

- **ESC's**: Electronic Speed Controller receives PWM control signal which is used by the Pixhawk flight controller to control motor speed by sending a series of digital pulses to the ESC. The ESC determines the throttle level based on the width of each pulse. It ranges from about 1 ms (low throttle) to 2 ms (high throttle).

- **Battery**: Using a 4s (s = number of cells) 4500 mAh LiPo battery with nominal voltage of 3.7 V per cell with total nominal voltage of 14.8 V and a peak voltage of 16.8 V. It has the capacity of 4.5A per hour (4500mAh). The drone draws a lot of current so we only get 12-15 minutes of flight time. It also has a discharge rating of 45C. The C rating tells you how many amps the battery can safely deliver. The C rating represents how quickly the battery can safely discharge relative to its capacity. For this battery, 1C corresponds to 4.5 A, meaning a 45C discharge rating theoretically allows up to 202.5 A of current.

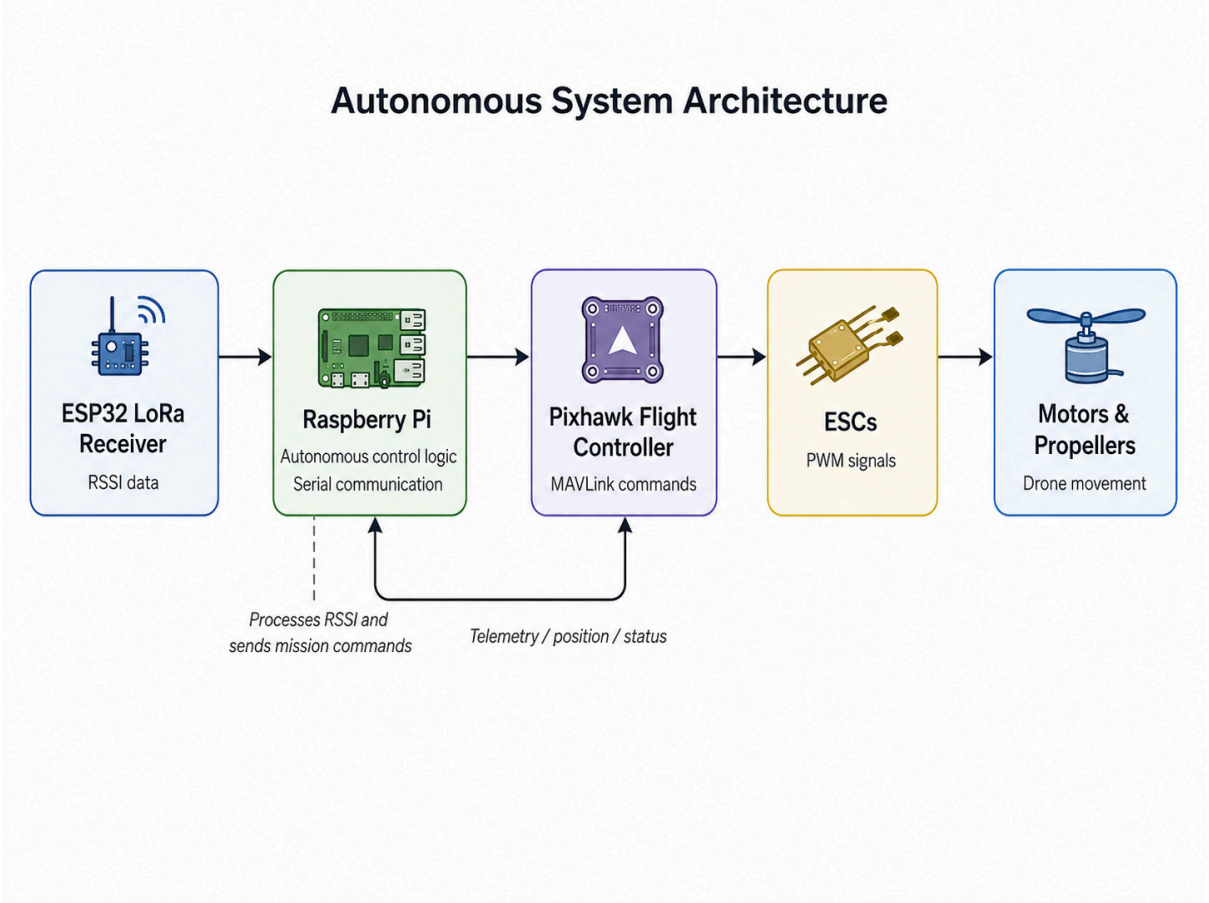
- **PIxhawk**: Pixhawk is a flight controller which connects to a GPS module and RC receiver. It also connects and controls the ESC's. We have to manually download and upload the flight controller firmware in Pixhawk which includes the navigation, stabilization, sensor processing, and other flight control functions.

- **Motors and Propellers**: We use 2212 motor size where the first 2 digits is the stator diameter (mm) and last 2 digits is stator length (mm). With that we have a KV rating (RPM per Volt) of 920 KV which means 920 RPM per volt under no load conditions . The KV rating corresponds to propeller size. As we have a higher KV rating we require a smaller propeller and vice versa.

- **Heltec LoRa transmitter and receiver**: The LoRa transmitter and receiver uses Esp32 microcontroller. The code is loaded inside the transmitter Esp32 and then we can connect a power bank to provide power to the transmitter. It sends RF packets/signals. The receiver measures the RSSI of those received packets and sends the RSSI value to the Raspberry Pi through one of the USB ports.

- **Raspberry Pi**: The Raspberry Pi is connected to the Pixhawk flight controller using the GPIO pins. It also connects to the Esp32 through a USB port. The Raspberry Pi runs the Autonomous Control logic and uses Pymavlink to communicate with the Pixhawk using the MAVLink protocol to run the autonomous program. The Raspberry Pi also receives the information from the ESP32 through the help of serial communication. Overall this works together to run the autonomous mission.

- **RC transmitter/receiver**: For manual flight only it is not used in the autonomous part unless we want to switch to manual flight if something goes wrong during the autonomous part. The RC transmitter used is the FlySky FS-i6 and receiver is FlySky FS-iA6B which is connected to the Pixhawk, which then controls the ESC's.



Autonomous Mission and Software Flow:

- The flow of the mission is shown below which is the task of test3.py.

  Creates Drone() Object → Guided() → Arm() → TakeOff() → Search() → Home() → Land()

  These functions are created in the drone3 file which contains the Drone() class. The drone class also makes use of the Location class from the file zigzag3.

- **__init__() :** This special method initializes the main variables used in the Drone class. In the function we create an object of the Location class and also activate the function called get_current_location () from the Location class to obtain the current latitude and longitude. These coordinates are saved inside the Drone class through variables. Later the variables are used to come back to Home base(Original location of UAV before take off).

- **GUIDED():** This function changes Pixhawk flight mode to GUIDED mode allowing autonomous actions which are controlled through Raspberry PI using MAVLInk protocol.

- **Arm() :** This function sends Mavlink arming command to Pixhawk. Arming the drone allows the drone to start the motors. Although before Arming the pre-arm safety checks must be satisfied.

- Takeoff(): This function sends a MAVLink takeoff command to the Pixhawk and includes a parameter for target altitude it needs to reach before continuing to search function.

- **Search():** This function makes use of a Location class object to run the functions inside it. We activate the zigzag function which controls the entire search mission. Functions below are defined inside the Location class (some other small functions are directly defined inside the coding files).

    - ZigZag(): This is the first function which is executed and it directs the drone in a particular pattern while monitoring the incoming RSSI values through the use of serial communication. The search pattern is composed of left , forward, right , forward movements. I chose this strategically to cover a larger area rather than going in a straight line. During each movement, the RSSI value is compared with a predetermined threshold. If the RSSI value exceeds the threshold indicating we are approaching the RF source we first stop the drone using stop() function. Then based on the direction it is facing it will activate the particular circle mode function. (e.g if we are moving left it will go to left_circle() function etc). The reason I choose a circle is because RSSI fluctuates due to things such as propagation, reflections and noise. Collecting multiple measurements around the surrounding area provides more information than relying on a single RSSI measurement additionally therefore providing a more robust estimate location of RF source.

- left_circle(): This function uses the X and Y coordinates that drone stopped at then based on it we use the for loop to generate angles every 15 degrees apart and we use the function x centre +rcos(θ) and y centre+rsin(θ) where r is the radius of the circle and θ theta is the angle in radians to complete the circle. At the same time it also records the RSSI values with its corresponding X and Y locations. These values are stored so that they can later be processed by the estimation_location function. Similar circle functions are used for the other directions of the zigzag pattern.

- **estimate_location:** This function makes use of the RSSI measurements and their corresponding X and Y coordinates collected during the circular search to estimate the location of the RF source.

  Each entry in the 2D measurement list contains an X position, Y position, and its corresponding RSSI value. The function first uses a for loop to search through the list and determine the strongest measured RSSI value, represented as RSSI_max. Each RSSI measurement is then converted into a relative weight using:

$$w_i = 10^{\frac{RSSI_i - RSSI_{\max}}{10}}$$

  This gives measurements with stronger RSSI values a larger influence on the final estimated location, while weaker measurements receive a smaller influence. The use of RSSI_max also makes the strongest measurement have a weight of 1, with the remaining measurements weighted relative to it.

  Additional for loops are then used to access the X position, Y position, and calculated weight from each entry in the 2D list. The estimated X and Y coordinates are calculated using a weighted average:

$$x_{\text{estimate}} = \frac{\sum_{i=1}^{n} x_i w_i}{\sum_{i=1}^{n} w_i}$$

$$y_{\text{estimate}} = \frac{\sum_{i=1}^{n} y_i w_i}{\sum_{i=1}^{n} w_i}$$

  Therefore, positions where stronger RSSI measurements were detected contribute more heavily to the final estimated source location. After

obtaining the estimated X and Y position, I used direct coordinate conversion formulas to convert these local displacement values into an estimated GPS latitude and longitude:

$$\text{Latitude}_{\text{reference}} = \text{GPS latitude at the local origin}$$

$$\text{Longitude}_{\text{reference}} = \text{GPS longitude at the local origin}$$

Then:

$$\text{Latitude}_{\text{estimate}} = \text{Latitude}_{\text{reference}} + \left(\frac{x_{\text{estimate}}}{R}\right)\frac{180}{\pi}$$

$$\text{Longitude}_{\text{estimate}} = \text{Longitude}_{\text{reference}} + \left(\frac{y_{\text{estimate}}}{R\cos\left(\text{Latitude}_{\text{reference}}\frac{\pi}{180}\right)}\right)\frac{180}{\pi}$$

where:

$$R = 6{,}371{,}000 \text{ m}$$

NOTE: The mathematical formula used for this estimation was derived with the assistance of AI, as developing a signal localization formula from first principles was beyond the scope of this project and the available time constraints. Although the mathematical weighting approach was provided through AI assistance, I implemented the translation of formula into code myself, including accessing and processing the 2D measurement list, calculating the individual weights, calculating the weighted X and Y values, and producing the final estimated coordinates.

- **Home():** This function makes use of the latitude and longitude variables that were saved during initialization and uses it in the parameters of the MAVlink position command. The drone then travels to the specified location based on the command and reaches home base.
- **Land():** This function sends the landing command to the Pixhawk, allowing the drone to automatically land after completing the mission.

Testing and validation:

LoRa communication test: verified that the transmitter and receiver communicated successfully and that RSSI values changed as the distance between them increased or decreased.

Serial communication test: confirmed that the ESP32 receiver transmitted RSSI readings to the Raspberry Pi through serial communication and that the Python program could read and process the values correctly.

Power distribution and soldering test: inspected and tested the soldered connections I did and power distribution system to confirm that power was being delivered correctly to the ESCs and motors without loose connections, shorts, or incorrect wiring.

Manual flight test: manually flew the drone before autonomous testing to verify motor response, stability, throttle control, and the basic operation of the Pixhawk, ESCs, motors, GPS, and flight hardware.

MAVLink communication test: confirmed heartbeat communication between the Raspberry Pi and Pixhawk and verified that the required position messages were being received.

SITL testing: tested GUIDED mode, takeoff, zigzag movement, circle logic, return to home, and landing in simulation before physical flight testing.

Physical zigzag test: confirmed that the drone could autonomously take off and execute the programmed zigzag search pattern.

Sequential movement validation: verified that the drone reached each target position before receiving the next movement command, preventing later commands from overriding previous targets.

RSSI threshold test: verified that exceeding the selected RSSI threshold caused the drone to stop the zigzag search and initiate circle mode.

Circle/estimation test: once the RSSI threshold was exceeded, the drone stopped its current search movement, entered the circular search pattern, and travelled through the target points while collecting RSSI measurements for source location estimation additionally checking whether the calculations gave valid estimation of RF source.

Return and landing test: successfully verified the return to home and landing sequence. After completing the search and estimation process, the drone navigated back to the saved starting GPS location and completed the landing sequence.

Problems & Challenges:

There were several problems and challenges throughout the project. One of the main problems was due to the Python script running much faster than the physical drone causing the python script to move to the next commands before the drone physically completes the current command. That caused the drone to stay motionless.To solve this, I implemented the target position logic explained earlier, where the drone's current position is continuously compared with the target position before allowing the program to continue.

Another problem occurred when transitioning from zigzag search pattern to circle mode. During the transition the drone would keep moving forward before the script reached the actual circle logic causing the circular pattern to not work properly. To improve the logic I created a stop function which stops the drone as soon as the RSSI exceeds the threshold. This fix made the transition smoother and maintained the action plan.

Another challenge occurred during some flight tests when the Pixhawk did not have a valid EKF position estimate, preventing the drone from arming and taking off. The EKF combines information from sensors such as the GPS and IMU to estimate the state and position of the aircraft. I learned that the system sometimes required additional time after initialization before a reliable position estimate was available, so I added a short delay before attempting to arm the drone. The physical test also introduced to challenges such as broken propellers , broken landing gears, lost propeller nuts which caused delays while the replacement parts were reordered but due to tight scheduling I had to replace the landing gear with an alternative plastic box which was strong enough to hold the drones weight and maintained balance while the drone is placed on the ground.

Overall, one of the most important lessons from this project was understanding that software and physical hardware must work together. Correct code alone does not guarantee correct system behaviour because physical movement, sensor timing, communication, and electrical components all affect how the complete system operates. I also realized that the testing process followed a similar idea to the V-model used in engineering design, where individual components were first tested separately, followed by integration testing and finally complete system testing.

Next Steps and improvements:

One improvement I would like to make in the future is to use a dynamic radius for the circular search pattern. In the current implementation, the circle uses a fixed radius of 2 metres. A possible improvement would be to use the specific RSSI value recorded after it exceeds the threshold to estimate the approximate distance between the drone and the RF source, then use this estimated distance to determine the radius of the circle. This could allow the circle pattern to adapt to the strength and get a better GPS estimate of the RF source with an improved recorded data during circle mode.

I would also like to develop a deeper understanding of MAVLink, particularly how its commands and messages are categorized, how the protocol is structured, and how communication between the Raspberry Pi and Pixhawk operates at a lower level. In addition, I would like to study LoRa communication in greater depth. Although this project introduced me to concepts such as operating frequency, bandwidth, spreading factor, coding rate, transmission power, and chirp spread spectrum, I would like to better understand how these parameters affect communication range, data rate, reliability, and received signal strength.

Conclusion:

This project successfully demonstrated an autonomous UAV system capable of detecting an RF source, estimating its location using RSSI measurements, and completing the mission through autonomous search, return to home, and landing. The project also provided valuable experience in integrating software, RF communication, flight control hardware, and physical system testing, which demonstrated the application of computer engineering in a real world system.

Project Demonstration:

Video demonstrations of the autonomous search pattern, RF signal detection, localization process, and return-to-home operation are available through the project links below.

Linked In: https://www.linkedin.com/in/dhruv-nikalwala/.
GitHub: https://github.com/Println-Dhruv/UAV-firmware/tree/main.

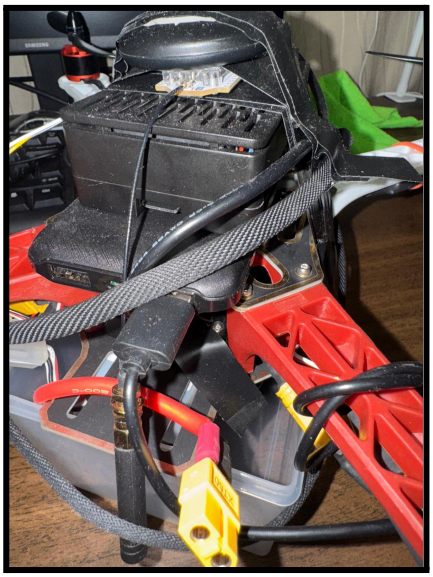
Reference:

- ArduPilot. (n.d.). *Copter commands in Guided Mode*. ArduPilot Developer Documentation. https://ardupilot.org/dev/docs/copter-commands-in-guided-mode.html
- ArduPilot. (n.d.). *Using SITL*. ArduPilot Developer Documentation. https://ardupilot.org/dev/docs/using-sitl-for-ardupilot-testing.html
- Heltec Automation. (n.d.). *Heltec LoRa basic library API*. GitHub. https://github.com/HelTecAutomation/Heltec_ESP32/blob/master/src/lora/API.md
- Heltec Automation. (n.d.). *WiFi LoRa 32 V3 factory test example*. GitHub. https://github.com/HelTecAutomation/Heltec_ESP32/blob/master/examples/Factory_Test/WiFi_LoRa_32_V3/WiFi_LoRa_32_V3_FactoryTest_V1/WiFi_LoRa_32_V3_FactoryTest_V1.ino
- MAVLink. (n.d.). *MAVLink common message set (common.xml)*. MAVLink Guide. https://mavlink.io/en/messages/common.html
- MAVLink. (n.d.). *Using Pymavlink libraries (mavgen)*. MAVLink Guide. https://mavlink.io/en/mavgen_python/
- Strang, G., & Herman, E. (2016). *Calculus volume 2*. OpenStax. https://openstax.org/details/books/calculus-volume-2
- Pochwat, M. (n.d.). *LiPo battery basics – How to choose the best one!* . YouTube. https://www.youtube.com/watch?v=EqXxr4YhXBU
- RCMakerLab. (n.d.). *RC brushless motors: What is KV? Motor size? Propeller size?* . YouTube. https://www.youtube.com/watch?v=hPa5yRCxnVA
- Painless360. (n.d.). *(1/5) PixHawk video series – Simple initial setup, config and calibration* YouTube. https://www.youtube.com/watch?v=uH2iCRA9G7k
- Alshaafal, F. (n.d.). *RF fundamentals part 1/3 – Learn all about radio frequency in 1 hour* . YouTube. https://www.youtube.com/watch?v=Rvti1TYI5NE
- YouTube. (n.d.). *Sound intensity and DeciBels*. https://www.youtube.com/watch?v=pCf28cxq6Wg
- Klonusk. (n.d.). *A brief guide to electromagnetic waves*. YouTube. https://www.youtube.com/watch?v=k3GQYBD5hbI

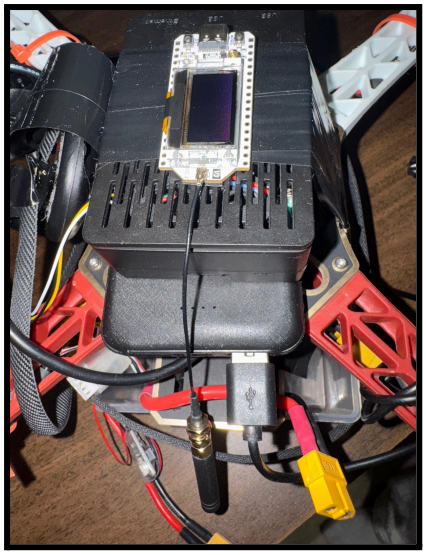
- Wermy. (2020, July 19). *Soldering crash course: Basic techniques, tips and advice!*. YouTube. https://www.youtube.com/watch?v=6rmErwU5E-k
- YouTube. (n.d.). *01 Pymavlink Connect* (entire series) https://www.youtube.com/watch?v=kecnaxlUiTY

APPENDIX :

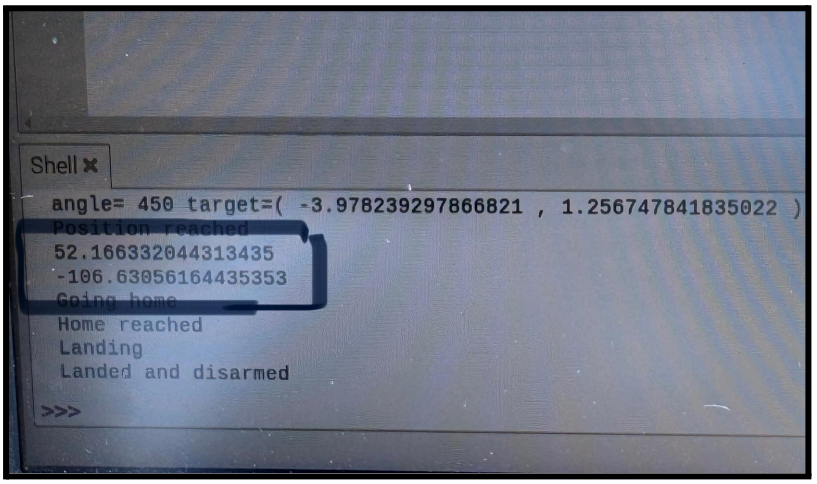
Appendix A: Drone Hardware Integration



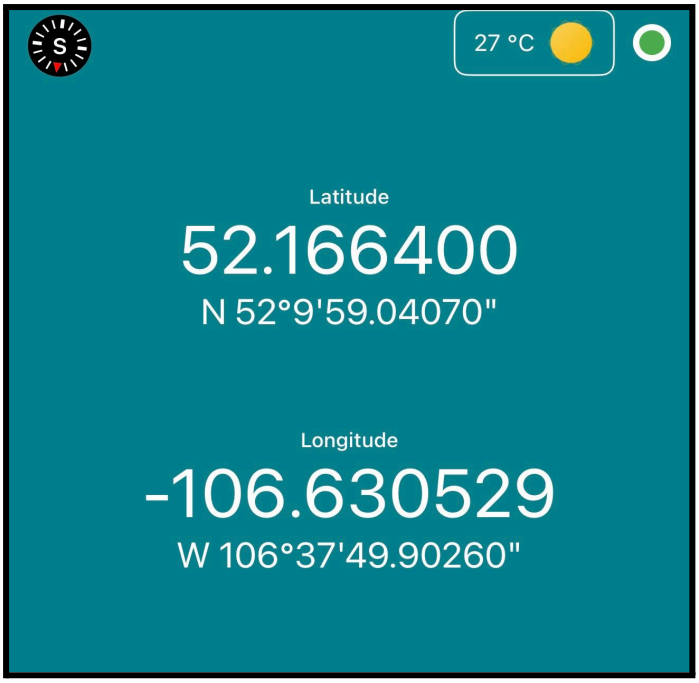
Appendix B – RF Detection Hardware



Appendix C – Localization Test Results



This image shows the GPS location estimated by the RSSI-based localization algorithm after the RF source was detected.



This image shows the actual GPS location of the RF source. The location was measured using a GPS application while standing directly above the RF transmitter.